

Praxis: Hintergrundarbeit mit Workqueues und Timern

Ein periodisches Kernelmodul entwickeln, beobachten und sicher stoppen

3.04

Ein vollständiger Arbeitszyklus

TIMER	Erzeugt alle period_ms ein Ereignis
WORKQUEUE	Verarbeitet das Ereignis im Prozesskontext
PARAMETER	Steuern Periode und simulierte Arbeitszeit
MESSWERTE	Zählen Ausführungen und Koaleszierungen
SHUTDOWN	Stoppt Timer und Work vor dem Entladen

Am Ende läuft das Modul periodisch und lässt sich sicher entladen.

Der Timer stößt an, der Worker verarbeitet

TIMER

**Softirq-Kontext
kurzer Callback**

SCHEDULE_WORK

**Queueing kann
koaleszieren**

WORKER

**Prozesskontext
Schlafen erlaubt**

Zeitpunkt und Verarbeitung bleiben bewusst getrennt.

Drei Dateien bilden das Experiment

WQT_DEMO.C

Modulzustand, Timer- und Work-Callbacks

MAKEFILE

obj-m += wqt_demo.o

RUN.SH

Build, insmod, Beobachtung und rmmod

VORAUSSETZUNG

Header passend zu uname -r installiert

MAKEFILE

```
obj-m += wqt_demo.o
```

```
all:
```

```
$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
```

Ein Container hält den Lebensdauervertrag

MODULZUSTAND

```
struct demo_ctx {
    struct timer_list timer;
    struct work_struct work;
    atomic_t runs;
    atomic_t coalesced;
    bool stopping;
};

static struct demo_ctx demo;
static unsigned int period_ms = 500;
static unsigned int work_ms = 100;
```

LEBENSDAUER

- Statischer Speicher
- Keine Freigabe vor module_exit

ZUGRIFFE

- READ_ONCE / WRITE_ONCE
- Atomic-Zähler

Container, Callbacks und Zähler teilen denselben Lebenszyklus.

Parameter machen Last sichtbar

MODULPARAMETER

```
module_param(period_ms, uint, 0644);  
MODULE_PARM_DESC(period_ms, "Timer period in ms");  
  
module_param(work_ms, uint, 0644);  
MODULE_PARM_DESC(work_ms, "Simulated work in ms");  
  
p = max_t(unsigned int, READ_ONCE(period_ms), 1);
```

NORMALBETRIEB

- period_ms = 500
- work_ms = 100

ÜBERLAST

- period_ms = 100
- work_ms = 350

Das Verhältnis aus Periode und Arbeitszeit erzeugt kontrollierte Last.

Der Worker darf blockierende Arbeit ausführen

WORKER

```
static void demo_work_fn(struct work_struct *work)
{
    struct demo_ctx *ctx = container_of(work, struct demo_ctx,
work);
    int n = atomic_inc_return(&ctx->runs);

    msleep(READ_ONCE(work_ms));
    pr_info("wqt_demo: run=%d cpu=%d\n", n, raw_smp_processor_id());
}
```

ERLAUBT

- msleep
- Mutex
- blockierendes I/O

BEGRENZEN

- Keine endlose Arbeit
- Shutdown muss warten können

Der Worker nutzt Prozesskontext, bleibt aber Teil des Shutdown-Vertrags.

Timer-Callback bleibt kurz und rearmt

TIMER CALLBACK

```
static void demo_timer_fn(struct timer_list *timer)
{
    struct demo_ctx *ctx = from_timer(ctx, timer, timer);
    unsigned int p;

    if (READ_ONCE(ctx->stopping))
        return;
    if (!schedule_work(&ctx->work))
        atomic_inc(&ctx->coalesced);

    p = max_t(unsigned int, READ_ONCE(period_ms), 1);
    mod_timer(&ctx->timer, jiffies + msecs_to_jiffies(p));
}
```

TIMER REGEL

- Markieren
- Queueen
- Rearmen

NICHT ERLAUBT

- msleep
- Mutex nehmen
- lange rechnen

Init richtet erst Work, dann Timer ein

MODULE_INIT

```
static int __init wqt_demo_init(void)
{
    if (!period_ms)
        return -EINVAL;

    atomic_set(&demo.runs, 0);
    atomic_set(&demo.coalesced, 0);
    WRITE_ONCE(demo.stopping, false);

    INIT_WORK(&demo.work, demo_work_fn);
    timer_setup(&demo.timer, demo_timer_fn, 0);
    mod_timer(&demo.timer, jiffies + msecs_to_jiffies(period_ms));
    return 0;
}

module_init(wqt_demo_init);
```

REIHENFOLGE

- Zustand
- Work Item
- Timer
- Start

VALIDIEREN

Ungültige Periode vor dem Start ablehnen.

Exit stoppt Producer vor Consumer

MODULE_EXIT

```
static void wqt_demo_exit(void)
{
    WRITE_ONCE(demo.stopping, true);
    timer_shutdown_sync(&demo.timer);
    cancel_work_sync(&demo.work);

    pr_info("wqt_demo: runs=%d coalesced=%d\n",
            atomic_read(&demo.runs),
            atomic_read(&demo.coalesced));
}

module_exit(wqt_demo_exit);
```

SICHER STOPPEN

- Producer sperren
- Timer stilllegen
- Work abwarten

ACHTUNG

Keine Locks halten, die der Worker benötigt.

Kbuild erzeugt das ladbare Modul

TERMINAL

```
$ make -C /lib/modules/$(uname -r)/build M=$PWD modules
$ modinfo ./wqt_demo.ko
$ sudo insmod ./wqt_demo.ko period_ms=500 work_ms=100
$ sudo dmesg -w
```

VORHER PRÜFEN

- `uname -r`
- Kernel-Header
- Secure Boot / Signatur

DANACH

```
sudo rmmod wqt_demo
```

Build, Laden und Entladen bilden den ersten vollständigen Test.

Logs zeigen wechselnde Worker-CPU's

BEOBACHTUNG

```
$ sudo dmesg -w  
[ 42.118] wqt_demo: run=1 cpu=3  
[ 42.619] wqt_demo: run=2 cpu=1  
[ 43.120] wqt_demo: run=3 cpu=1  
  
$ cat /sys/module/wqt_demo/parameters/period_ms  
$ cat /sys/module/wqt_demo/parameters/work_ms
```

BEOBACHTEN

- Abstand der Meldungen
- CPU des Workers
- Wirkung der Parameter

NICHT ABLEITEN

- Keine garantierte Deadline
- Keine feste CPU-Zuordnung

Logs zeigen Ausführung, aber keine harte Zeit- oder CPU-Garantie.

Langsame Work macht Koaleszierung sichtbar

LASTWECHSEL

```
$ echo 100 | sudo tee  
/sys/module/wqt_demo/parameters/period_ms  
$ echo 350 | sudo tee /sys/module/wqt_demo/parameters/work_ms  
$ sleep 5  
$ sudo rmmod wqt_demo  
  
# Ausgabe: runs=... coalesced=...
```

ERWARTUNG

- runs wächst langsamer
- coalesced wird größer

BEDEUTUNG

Das Work Item signalisiert Arbeit und bildet keine Ereignisqueue.

Koaleszierung wird sichtbar, wenn Arbeit länger als die Periode dauert.

Tracing bestätigt Timer und Workqueue

TRACE-CMD

```
$ sudo trace-cmd record -e workqueue -e timer sleep 5
```

```
$ sudo trace-cmd report
```

```
# Verfügbare Events prüfen:
```

```
$ ls /sys/kernel/tracing/events/workqueue
```

```
$ ls /sys/kernel/tracing/events/timer
```

FRAGEN

- Wann queued?
- Wann gestartet?
- Auf welcher CPU?
- Wie lange lief die Work?

ABGLEICH

Logmeldungen mit Trace-Ereignissen verbinden.

Tracing trennt Queueing-Zeitpunkt und tatsächliche Ausführung.

Delayed Work vereinfacht periodische Jobs

DELAYED WORK

```
static void poll_fn(struct work_struct *work)
{
    struct demo_ctx *ctx = container_of(
        to_delayed_work(work), struct demo_ctx, dwork);

    read_and_process(ctx);
    schedule_delayed_work(&ctx->dwork,
        msecs_to_jiffies(period_ms));
}

INIT_DELAYED_WORK(&demo.dwork, poll_fn);
schedule_delayed_work(&demo.dwork, 0);
```

VERWENDUNG

- Retry
- Polling
- Debounce

SHUTDOWN

cancel_delayed_work_sync()

Das Modul gezielt erweitern

01 Statusmeldung

runs und coalesced alle zehn Ausführungen loggen

02 Validierung

Parameter begrenzen und ungültige Werte ablehnen

03 Fehlerpfad

Init-Fehler nachstellen und Cleanup prüfen

04 Lastwechsel

Parameter während der Laufzeit ändern

05 Variante

Timer durch `delayed_work` ersetzen

Testfolge: Build, Laden, Beobachten, Überlast, Entladen und Wiederholen.